

Lab 3: Implementation of a Feedforward Neural Network for Classification

Course-	M.Tech.	Type- Core	Core
Course Code-	ABAI5005L	Course Name-	Advanced Machine Learning
Year-	2025-26	Semester-	ODD
Date-	04-08-2026	Batch-	1

*Scope of this lab: This lab focuses only on **Feedforward Neural Networks (Multi-Layer Perceptron)** for classification tasks. For each question, build, train, evaluate, and interpret the model. Duration: 2 Hours. Software: Python 3.x, Jupyter Notebook / Google Colab.*

Learning Objectives

After completing this laboratory, students will be able to:

- Understand the architecture of a feedforward neural network.
- Prepare and preprocess classification datasets.
- Build a neural network using TensorFlow/Keras.
- Train and validate a neural network model.
- Evaluate classification performance using different metrics.
- Study the effect of hyperparameters such as epochs, learning rate, and hidden neurons.

Background

Artificial Neural Networks (ANNs) are inspired by biological neurons. A typical feedforward neural network consists of an Input Layer, one or more Hidden Layers, and an Output Layer.

Each neuron computes: $y = f(\sum w_i x_i + b)$, where x_i is the input, w_i is the weight, b is the bias, and f is the activation function.

For classification problems, hidden layers usually use ReLU, while the output layer uses Sigmoid for binary classification or Softmax for multi-class classification.

Dataset

Use any one of the following datasets throughout this lab.

Option	Dataset	Target Classes
Option 1 (Recommended)	Breast Cancer Wisconsin Dataset — <code>sklearn.datasets.load_breast_cancer()</code>	Malignant, Benign
Option 2	Iris Dataset — <code>sklearn.datasets.load_iris()</code>	Setosa, Versicolor, Virginica

Required Libraries

Library	Purpose
---------	---------

numpy	Numerical computing
pandas	Data handling
matplotlib.pyplot	Visualization
sklearn.datasets	Load datasets
sklearn.model_selection	Train-test split
sklearn.preprocessing	Feature scaling
sklearn.metrics	Model evaluation
tensorflow.keras	Neural network implementation

Question 1: Dataset Loading and Exploration

Objective: Load the chosen classification dataset and explore its structure before modelling.

Instructions:

1. Load the Breast Cancer (or Iris) dataset.
2. Display the first five samples.
3. Display the dataset shape.
4. Display the number of features.
5. Display the number of classes.
6. Check for missing values.

Python / Keras hints:

- `load_breast_cancer()` / `load_iris()` to load the dataset
- `pandas.DataFrame()` to view the data as a table
- `head()`, `info()`, `describe()` for dataset exploration
- `shape` and `columns` to inspect dimensions and feature names

Question 2: Data Preprocessing

Objective: Prepare the dataset for neural network training.

Instructions:

7. Split the dataset into training and testing sets.
8. Standardize all numerical features.
9. Explain why feature scaling is important for neural networks.

Python / Keras hints:

- `sklearn.model_selection.train_test_split(X, y, test_size=?, random_state=?)` to split the dataset
- `sklearn.preprocessing.StandardScaler()` — `fit()`, `transform()`, `fit_transform()` to normalize features

Question 3: Building and Training a Feedforward Neural Network

Objective: Construct and train a feedforward neural network (MLP) for classification.

Instructions:

10. Build a network with an input layer, at least one hidden layer, and an output layer.

11. Use ReLU activation in the hidden layer(s).
12. Use Sigmoid (binary) or Softmax (multi-class) activation in the output layer.
13. Compile the model with a suitable optimizer, loss function, and metrics.
14. Train the model and record the Training Accuracy, Validation Accuracy, and Loss Curve.

Python / Keras hints:

- Sequential() to create the network; Dense(units=?, activation=?) to add layers
- compile(optimizer=?, loss=?, metrics=?) — optimizers: adam, SGD, RMSprop
- Loss functions: binary_crossentropy (binary); categorical_crossentropy or sparse_categorical_crossentropy (multi-class)
- fit(X_train, y_train, epochs=?, batch_size=?, validation_split=?) to train the model

Question 4: Model Evaluation

Objective: Evaluate the trained model on the test set and interpret the results.

Instructions:

15. Calculate the Test Accuracy.
16. Compute the Confusion Matrix.
17. Compute Precision, Recall, and F1-score.
18. Interpret the results.

Python / Keras hints:

- predict() to generate predictions; evaluate() to score the model on test data
- accuracy_score(), confusion_matrix(), classification_report() for classification metrics

Question 5: Effect of Network Architecture and Training Epochs

Objective: Study how the number of hidden layers and training epochs affect model performance.

Instructions:

19. Create a neural network with one hidden layer and train it using 20 epochs. Record the Training and Testing Accuracy.
20. Create another model with two hidden layers. Compare its Accuracy and Training Time against the first model. Which architecture performs better?
21. Using the same model, train with 10, 50, and 100 epochs and complete the table below.
22. Discuss whether increasing the number of epochs always improves performance.

Results Table:

Epochs	Training Accuracy	Validation Accuracy	Test Accuracy
10			
50			
100			

Python / Keras hints:

- Reuse Sequential()/Dense() to add a second hidden layer for comparison
- time module or %%time (Colab/Jupyter) to measure training time
- History object returned by fit() to plot accuracy/loss vs. epochs

Question 6: Effect of Optimizer Choice

Objective: Compare the effect of different optimizers on training the same neural network.

Instructions:

23. Train the same neural network using the Adam optimizer.
24. Train the same neural network using the SGD optimizer.
25. Train the same neural network using the RMSprop optimizer.
26. Compare Accuracy, Convergence speed, and Final loss across the three optimizers.
27. Discuss which optimizer is most suitable for this dataset.

Python / Keras hints:

- `compile(optimizer='adam' / 'sgd' / 'rmsprop', loss=?, metrics=?)`
- Compare the loss/accuracy curves returned in the History object for each optimizer

Expected Output: Students should include a dataset description, network architecture summary, training history plots (Accuracy and Loss), test accuracy, confusion matrix, classification report, comparative analysis for different hyperparameters, and a discussion on the impact of network architecture, epochs, and optimizers.